DLS

Alignment of Diffusion Models

Deep Learning School · RLHF module



ABOUT THE SPEAKER

Lev Novitskiy

Video Post-train · Kandinsky Lab

I work on post-training of diffusion models for video generation

EXPERIENCE	5+ years in ML
STACK	PyTorch
CONTACT	@lefffffffffff levnovitskiy@gmail.com

@MLBALL_DAYS

Roadmap

DLS

01

The problem

Why a pretrained model is not yet what users want

02

SFT & soups

Curated, balanced data; averaging per-domain fine-tunes

03

RL framework

Prompts, rollouts, rewards, losses; reward hacking

04

Offline vs online

Fixed datasets vs. learning from your own samples

05

Differentiable RL

ReFL, DRaFT, DRaFT-K, DRaFT-LV

06

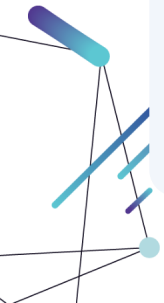
Policy gradient & beyond

REINFORCE → PPO → DDPO, DPO, GRPO family; then DiffusionNFT

07

Merging rewards

On-policy distillation (OPD) of reward experts



The problem

Part 1

Pretraining teaches a model to imitate the internet, not to do what we want

Motivation: the pretrained model generates poorly

STARTING POINT

Pretrained model

Fits p(x | c) on billions of noisy web pairs → imitates the *average* of the internet

- Weak prompt following
- Garbled text
- Mediocre aesthetics, bad hands
- Unsafe or biased outputs

→

STEP 1 · SFT

Show what good looks like

Fine-tune on a small, curated, balanced set of ideal examples

Signal: imitation of positives only

Fixes: style, aesthetics, caption format

→

STEP 2 · RL

Tell how good each sample is

A reward scores the model's *own* samples; the model moves toward high reward

Signal: positives and negatives

Fixes: prompt following, text, preferences

Alignment = post-training that moves the model from "what the web looks like" to "what users want".

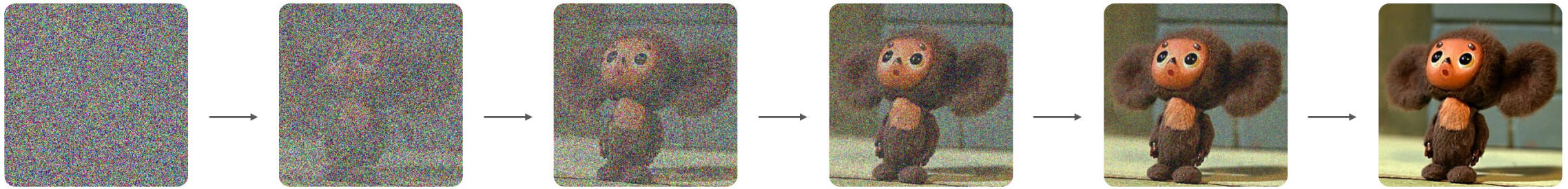
Recap: diffusion and flow models

FORWARD (NOISING) PROCESS

$$x_t = \alpha_t\, x_0 + \sigma_t\, \epsilon, \qquad \epsilon \sim \mathcal{N}(0, I)$$

SAMPLING: REVERSE ODE / SDE

$$\frac{dx_t}{dt} = v_\theta(x_t, t, c), \qquad t : 1 \to 0 \quad \text{(deterministic sampler)}$$



Generation = denoising: start from pure noise (t = 1) and integrate the learned field to the image (t = 0), 20–50 network calls

Training losses: what the network predicts

E-PREDICTION (DDPM, SD 1.X)

$$\mathcal{L}_{\mathrm{DM}}(\theta) = \mathbb{E}_{x_0,c,\epsilon,t} \left\| \epsilon - \epsilon_\theta(x_t, t, c) \right\|^2$$

X_0-PREDICTION

$$\mathcal{L}_{x_0} = \mathbb{E}_{x_0,\epsilon,t} \left\| x_0 - \hat{x}_\theta(x_t, t, c) \right\|^2$$

V-PREDICTION (SALIMANS & HO, 2022; SD 2.X)

$$v_t = \alpha_t\, \epsilon - \sigma_t\, x_0, \qquad \mathcal{L}_v = \mathbb{E}_{x_0,\epsilon,t} \left\| v_t - v_\theta(x_t, t, c) \right\|^2$$

FLOW MATCHING (SD3, FLUX, KANDINSKY 5) — VELOCITY OF THE PATH

$$x_t = \alpha_t x_0 + \sigma_t \epsilon, \quad \frac{dx_t}{dt} = \dot{\alpha}_t x_0 + \dot{\sigma}_t \epsilon, \qquad \mathcal{L}_{\mathrm{FM}} = \mathbb{E}_{x_0,\epsilon,t} \left\| v_\theta(x_t, t, c) - (\dot{\alpha}_t x_0 + \dot{\sigma}_t \epsilon) \right\|^2$$

$$\mathcal{L}_w(\theta) = \mathbb{E}_t \left[w(t)\, \mathbb{E}_{x_0,\epsilon} \left\| \epsilon - \hat{\epsilon}_\theta(x_t, t, c) \right\|^2 \right]$$

One family: all four are the same denoising objective with a different time weighting w(t) — a weighted ELBO (Kingma & Gao, 2023). Everything is written for a generic schedule $x_t = \alpha_t x_0 + \sigma_t \epsilon$.

Why diffusion is harder to align than LLMs

Aspect	LLMs	Diffusion / flow models
Likelihood	Exact log p(y|x) as a sum of token log-probs	log $p(x_0|c)$ intractable — only the ELBO or per-step Gaussians
“Sequence”	Tokens are the content itself	T denoising steps over a latent trajectory, noise → x_0
Action space	Discrete vocabulary	Continuous, high-dimensional Gaussian transitions
Sampler	Stochastic sampling is native	Best samplers are deterministic ODEs — no policy density
Rollout cost	One forward pass per token (KV cache)	20–50 network passes per sample, ×2 with CFG; video far worse
Reward	Verifiers, reward models	Preference models, OCR, detectors, VLM judges — often differentiable

SFT & model soups

Part 2

Fine-tuning on small, curated, balanced data — and averaging per-domain fine-tunes

SFT: quality tuning on curated data

DLS

SAME LOSS AS PRETRAINING, DIFFERENT DATA

$$\mathcal{L}_{\text{SFT}}(\theta) = \mathbb{E}_{(x_0,c)\sim\mathcal{D}_{\text{curated}}}\, \mathbb{E}_{t,\epsilon}\left\|\epsilon - \epsilon_\theta(x_t, t, c)\right\|^2$$

Quality over quantity: Emu quality-tuned on only a few thousand hand-picked photos → 82.9% win rate vs. the pretrained model

Balanced dataset: even coverage of domains (people, animals, food, text, art…) — otherwise SFT drags the model toward over-represented content

Expensive captions are affordable: the set is small, so we can caption it with large VLMs, several caption lengths, even human checks — too costly for the full pretraining corpus

1 · Automatic filters

aesthetic / quality scores, watermarks, NSFW

2 · Human experts + domain balance

keep a few %; sample domains evenly

3 · Rich captions → fine-tune

large-VLM captions, several lengths



Dai et al., Emu — arXiv:2309.15807

SFT in practice: Kandinsky 5.0 and 6.0

Kandinsky 5.0 Image

- ~10M high-res images → two-stage human review (annotators, then visual-arts experts) → ~5% kept
- Strict v1: 45k images; relaxed v2: 153k
- 9 domains assigned by Qwen2.5-VL-32B; sampled with calibrated probabilities
- Captions: Qwen2.5-VL-32B description, rewritten by Qwen3-32B into 5 lengths, EN + RU

Kandinsky 5.0 Video

- ~93k high-res scenes → same two-stage review → ~3% kept
- Strict v1: 2,833 videos; relaxed v2: 12,461
- Same 9 domains as images
- Captions: SkyCaptioner-V1 + Qwen2.5-VL 32B/72B
- Plain SFT overfits after ~10k steps

Kandinsky 6.0 Video

- 11 domains, two SFT stages
- Stage 1, visual quality: 4,663 scenes
- Stage 2, audio–video sync & lip-sync: 7,686 scenes
- Filters: DOVER, Q-Align, Audiobox Aesthetics; experts check lip-sync
- Joint video + audio captions (Qwen3.5-35B-A3B, Qwen3-Omni), 4 detail levels

Kandinsky 5.0 — arXiv:2511.14993 · Kandinsky 6.0 Video — arXiv:2610.05608

SFT: strengths and limits

Strengths

Stable: plain regression loss, no reward model, no sampling loop

Cheap: one forward/backward per example, like pretraining

Controllable: you see exactly which images the model imitates

Limits

Only positives: no signal from what is bad

Ceiling = data: cannot exceed the curated set

Off-policy: trains on someone else's images, not the model's own mistakes

Expensive curation and **diversity loss** on small sets

Model soups: averaging fine-tuned weights

DLS

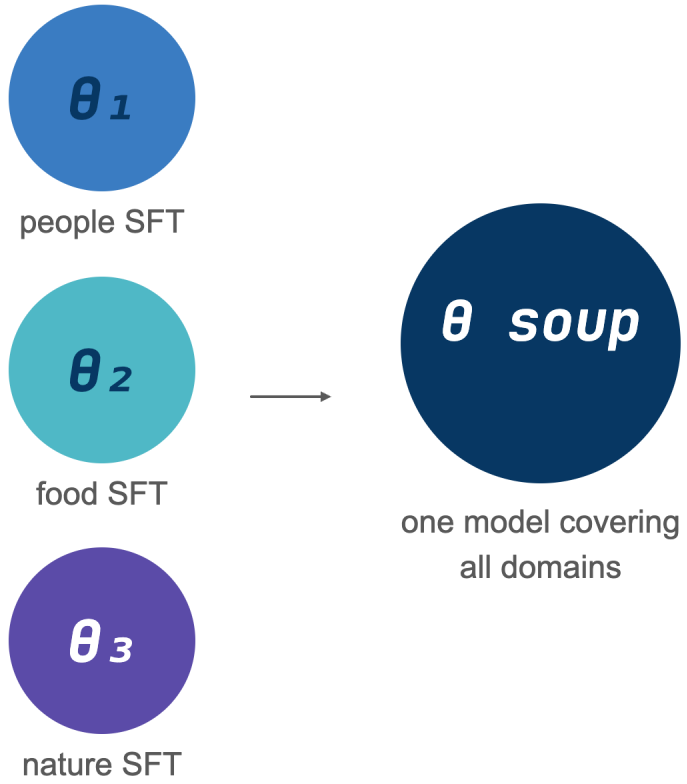
WEIGHT AVERAGING

$$\theta_{\text{soup}} = \sum_{i=1}^{K} \lambda_i \, \theta_i, \qquad \lambda_i \geq 0, \quad \sum_i \lambda_i = 1$$

Same init is essential: all ingredients start from the same base weights → linear mode connectivity

Uniform soup: $\lambda_i = 1/K$

Free at inference: one model, no ensemble cost



Wortsman et al., Model soups, ICML 2022 — arXiv:2203.05482 · Ramé et al., Rewarded soups, NeurIPS 2023 — arXiv:2306.04488

SFT soups in practice: Kandinsky 5.0 and 6.0

Kandinsky 5.0 Image

Problem: full SFT on all 153k images kept a non-photoreal style and hurt text alignment and rendering

- 9 VLM domains, each split into 2–9 subdomains
- One full fine-tune per subdomain (LR 1e-5, stopped when artifacts appear)
- Merge within a domain, weights $\propto \sqrt{(\text{subdomain size})}$; then average the 9 domain models
- Hierarchical soup beat VLM-only and CLIP k-means clusters in SBS

Kandinsky 5.0 Video

Problem: standard SFT overfits after ~10k steps

- One model per each of the 9 domains
- Uniform weight average
- Internal SBS: soup beats standard SFT on visual quality, motion consistency and artifacts, with equal text alignment

Kandinsky 6.0 Video

Setup: 11 domains, two SFT stages

- Each domain model trained independently: Stage 1 visual quality (7k steps), Stage 2 audio–video sync & lip-sync (5k steps)
- Stage 2 continues from the domain's own Stage 1 checkpoint
- 11 checkpoints merged by uniform averaging only at the end of Stage 2

Kandinsky 5.0 — arXiv:2511.14993 (Sec. 4.6, 6.4) · Kandinsky 6.0 Video — arXiv:2610.05608 (Sec. 4.3)

RL framework & rewards

Part 3

Two independent design choices: what to optimize, and how

Why RL after SFT?

Judging is easier than creating: a preference click, an OCR check or a detector is cheap — an ideal image for every prompt is not

Learns from its own samples: including the bad ones, which SFT never sees

Any measurable goal: text accuracy, object counts, aesthetics, safety

Can exceed the data: limited by the reward, not by the curated set

WHAT AN RL LOOP IS MADE OF

1. **Prompts**
   which conditions we train on
2. **Rollout**
   who generates the samples, and how
3. **Reward**
   how good each sample is
4. **Loss / update**
   how the reward reaches the weights, while staying close to the SFT model

Next: the reward first, then rollouts and losses.

Constructing a reward

Human feedback (HF)

Human Preference Score: HPS v2 / v2.1 / v3 — CLIP/VLM fine-tuned on human choices

ImageReward: BLIP-based RM from expert rankings (also introduced ReFL)

PickScore: CLIP-H trained on Pick-a-Pic user preferences

Aesthetic predictors: LAION aesthetic MLP on CLIP features

Video: VideoAlign / VideoScore for quality and motion

Others (rule / model-based)

Text rendering (OCR): run OCR, compare with the target string

Prompt following (CLIP, CLAP): prompt–image (CLIP) or prompt–audio (CLAP) similarity

Compositional (GenEval): detector checks counts, colors, positions

VLM-as-judge: VQAScore, UnifiedReward

Toy rewards: JPEG compressibility (DDPO)

Learning a reward from human preferences

DLS

BRADLEY–TERRY MODEL

$$P\left(x^w \succ x^l \,\middle|\, c\right) = \sigma\left(r_\phi(x^w, c) - r_\phi(x^l, c)\right)$$

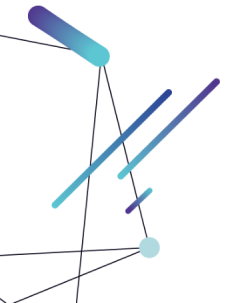
REWARD-MODEL LOSS

$$\mathcal{L}_{\mathrm{RM}}(\phi) = -\,\mathbb{E}_{(c,\,x^w,\,x^l)\sim\mathcal{D}}\,\log\sigma\left(r_\phi(x^w, c) - r_\phi(x^l, c)\right)$$

Pitfall: never report only the reward you trained on — use held-out RMs and human eval

Reward model	Backbone	Data
ImageReward (2023)	BLIP	~137k expert comparisons
PickScore (2023)	CLIP-H	Pick-a-Pic web-user choices
HPS v2 / v2.1 (2023)	CLIP-H	HPD v2: ~798k choices
HPS v3 (2025)	VLM	HPDv3: ~1.08M images, ~1.17M pairs

Dataset sizes as reported by the papers (rounded).



Rule-based and model-based rewards

DLS

TEXT RENDERING (OCR) · N_e = EDIT DISTANCE

$$r_{\mathrm{OCR}}(x_0, c) = \max\left(1 - \frac{N_e}{N_{\mathrm{ref}}},\ 0\right)$$

PROMPT FOLLOWING (CLIP / CLAP)

$$r_{\mathrm{CLIP}}(x_0, c) = \cos\left(E_{\mathrm{img}}(x_0),\ E_{\mathrm{txt}}(c)\right)$$

GenEval reward

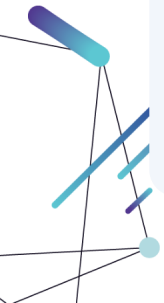
A detector verifies object count, color, position and co-occurrence → fraction of satisfied constraints

VLM-as-judge

Ask a VLM “Does this image show …?” — P(yes) as reward (VQAScore), or a fine-tuned judge (UnifiedReward)

Binary / thresholded

Turn any score into 0/1 success — robust to scale, harder to hack



Reward hacking

Symptoms

Aesthetic / HPS
Oversaturated, extreme contrast, one style for every prompt

PickScore / CLIP
Prompt words pasted into the image, less diversity

OCR
Text appears even where not requested

Any single RM
Mode collapse: every seed gives nearly the same picture →



Example of mode collapse — DDPO (Black et al., 2023)

Mitigations

KL to reference: the β term, closed form for Gaussian policies

Small updates: LoRA, low LR, PPO-style clipping

Early stopping: held-out reward models and human eval

Shared noise in group: fair comparison of samples

Multiple rewards: HF + rule-based, no single proxy dominates

Anchor regularization: pull toward a quality teacher (Flow-OPD)

Rollout and loss: the other two components

Rollout — where samples come from

Who generates: a fixed dataset (offline) or the current model (online)

Sampler: deterministic ODE vs. stochastic SDE — likelihood-based losses need the SDE

Group size: several samples per prompt to compare them

Cost knobs: fewer denoising steps, shared initial noise, CFG on/off

Loss — how the reward reaches the weights

Differentiable RL: follow ∇r through the sampler — ReFL, DRaFT

Policy gradient: $r \times \nabla \log p$ — REINFORCE, RWR, DDPO, GRPO family

Preference pairs: Diffusion-DPO

No policy gradient: DiffusionNFT (forward process), OPD (distillation)

Plus a regularizer: KL to the SFT model

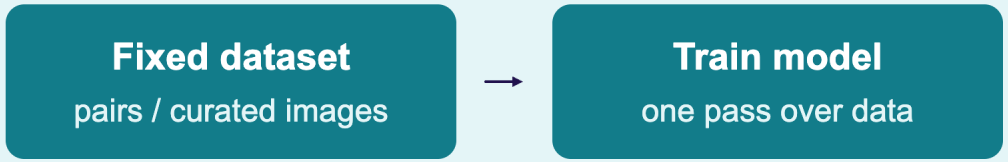
Offline vs online

Part 4

Learning from a fixed dataset vs. learning from your own samples

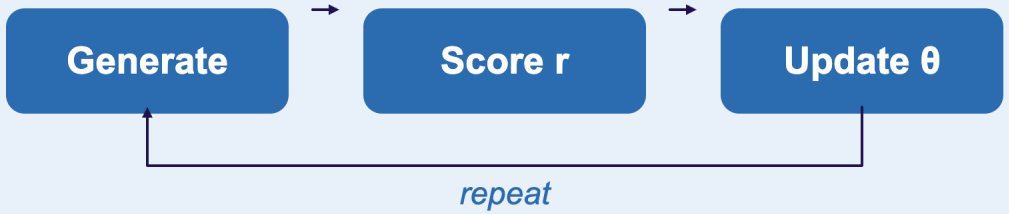
RL offline vs online

Offline



- Data collected once, by others
- Cheap, simple, reproducible
- Distribution shift: never sees its own mistakes
- **Examples:** SFT, RWR, Diffusion-DPO

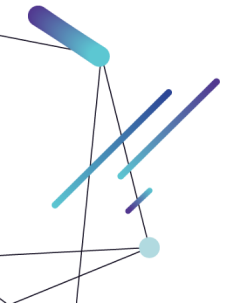
Online



- Samples come from the current policy
- Learns from its own mistakes; explores
- Costly: sampling inside the training loop
- **Examples:** ReFL, DRaFT, DDPO, Flow-GRPO, DiffusionNFT, OPD

Offline vs online: trade-offs

	Offline	Online
Data source	Fixed: humans, other models, web	Current model's own samples
Reward model	Not needed (pairs or curated images)	Queried during training
Distribution shift	Yes — off-policy data	No — on-policy by construction
Compute	≈ fine-tuning cost	Fine-tuning + sampling each iteration
Main risk	Overfitting, weak gains	Reward hacking, instability
Methods	SFT, RWR, Diffusion-DPO	ReFL, DRaFT, DDPO, Flow-GRPO, NFT, OPD



Differentiable RL

Part 5

If the reward is differentiable, follow its gradient through the sampler

Direct reward backpropagation

DLS

CHAIN RULE THROUGH THE SAMPLER

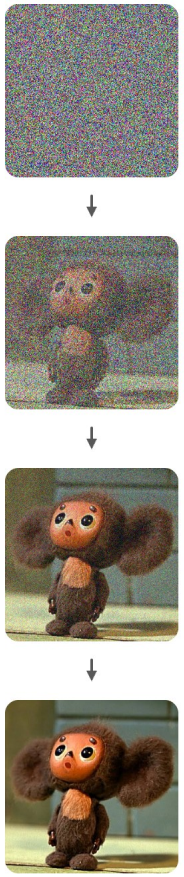
$$\nabla_\theta\, r\big(x_0(\theta), c\big) = \frac{\partial r}{\partial x_0} \cdot \frac{\partial x_0}{\partial \theta}, \qquad x_0 = \mathrm{Sampler}_\theta(x_T, c)$$

Requirement: r must be differentiable (RMs, CLIP: yes; OCR engines, detectors: often no)

Memory: activations for all T steps → gradient checkpointing, LoRA

Stability: product of T Jacobians → exploding / vanishing gradients

Payoff: the full gradient of r, not one scalar → very sample-efficient



↑ ∇r flows back through every denoising step

r(x_0, c)

ReFL — Reward Feedback Learning (ImageReward)

DLS

ONE-STEP PREDICTION OF X_0

$$\hat{x}_0 = \frac{x_t - \sigma_t \, \hat{\epsilon}_\theta(x_t, t, c)}{\alpha_t}$$

LOSS

$$\mathcal{L}_{\text{ReFL}} = \lambda \, \mathbb{E}_c\big[\phi\big(r(\hat{x}_0, c)\big)\big] + \mathcal{L}_{\text{pre}}, \qquad t \sim \mathcal{U}[T_1, T_2]$$

1 · Pick step
t ~ U[T_1, T_2] among the last steps

→

2 · No-grad rollout
noise → x_t with the frozen sampler

→

3 · One step with grad
predict $\hat{x}_0$ directly (“jump”)

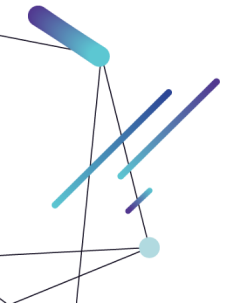
→

4 · Update
ReFL loss + $\mathcal{L}$_pre

φ — reward-to-loss map
Turns the reward into a loss to minimize; the paper sets φ = ReLU and λ = 10^{-3}

$\mathcal{L}$_pre — pretraining loss
The ordinary denoising loss on pretraining data (625k aesthetic subset of LAION-5B), half of each batch — keeps the model from drifting and overfitting the RM

Cons: only the late step gets credit; $\hat{x}_0$ is an approximation, blurry if t is not late enough



DRaFT – Direct Reward Fine-Tuning

DLS

EXACT GRADIENT THROUGH ALL SAMPLING STEPS

$$\mathcal{L}_{\mathrm{DRaFT}}(\theta) = -\,\mathbb{E}_{c,\,x_T}\Big[r\big(\mathrm{sample}(\theta, x_T, c),\ c\big)\Big]$$

No φ, no $\mathcal{L}$_pre: the loss is simply −r; drift is controlled by LoRA scaling and early stopping instead

Gradient checkpointing: store only latents x_t, recompute activations on backward

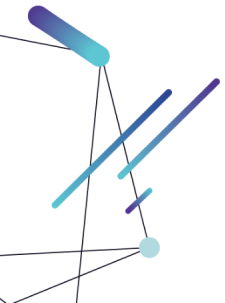
Observation: reward rises within a few hundred steps — and so does hacking

LoRA: only low-rank adapters are trained → small memory, easy to scale or mix

Rewards: LAION aesthetic, HPSv2, PickScore, compressibility

Problem: gradients through all T steps are costly and noisy → DRaFT-K, DRaFT-LV

Clark et al., ICLR 2024 — arXiv:2309.17400



DRaFT-K and DRaFT-LV

DRAFT-K — BACKPROP ONLY THROUGH THE LAST K STEPS (K = 1 IS OFTEN BEST)

$$\nabla_\theta r(x_0) \approx \sum_{t=1}^{K} \frac{\partial r}{\partial x_0} \frac{\partial x_0}{\partial x_{t-1}} \frac{\partial x_{t-1}}{\partial \theta} \qquad \text{(stop-grad before step } K\text{)}$$

DRAFT-LV ("LOW VARIANCE") = DRAFT-1 WITH N RE-NOISED COPIES OF THE LAST STEP

1 · Sample x_0
full chain, no gradients (expensive, done once)

→

2 · Re-noise n times
$x_1^{(i)}$ = noised x_0 with fresh noise ε_i

→

3 · One step with grad
denoise each $x_1^{(i)}$ → $x_0^{(i)}$ (cheap)

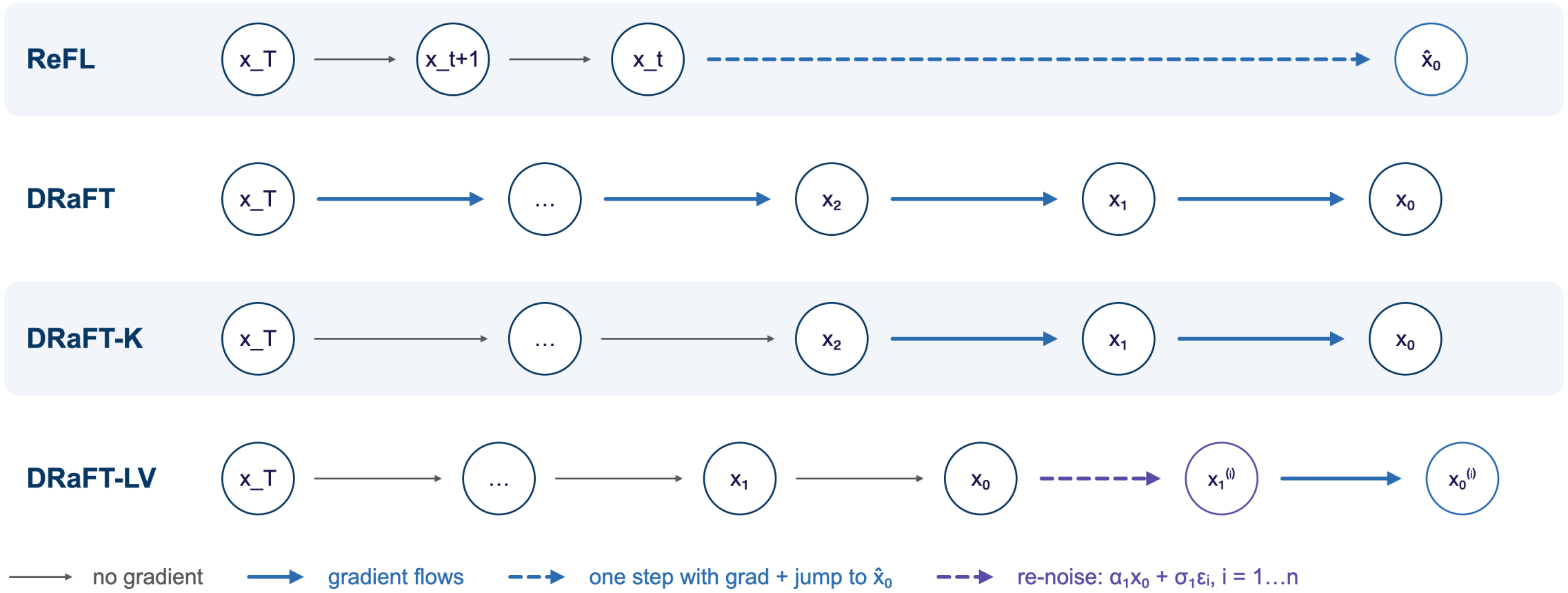
→

4 · Average
mean of n reward gradients

$$x_1^{(i)} = \alpha_1 \, \mathrm{sg}(x_0) + \sigma_1 \, \epsilon_i, \qquad \nabla_\theta \approx \frac{1}{n} \sum_{i=1}^{n} \nabla_\theta \, r\big(\mathrm{step}_\theta(x_1^{(i)})\big)$$

Why it helps: one expensive rollout is shared by n cheap gradient estimates — averaging them cuts the noise of a single one-step gradient. The most sample-efficient variant in the paper.

Where the gradient flows



Differentiable RL: when to use it

	ReFL	DRaFT	DRaFT-K	DRaFT-LV
Steps with gradient	1 (random late t)	All T	Last K	Last 1, ×n noises
Uses true x_0?	No ($\hat{x}_0$ jump)	Yes	Yes	Yes
Memory	Low	High	Low–medium	Low
Gradient variance	Medium	High	Lower	Lowest
Hacking risk	Medium	High	Medium	Medium

Use when

The reward is a differentiable network (HPS, PickScore, aesthetic) and you want fast, cheap improvement

Avoid when

The reward is a black box (OCR engine, detector, VLM text) or hacking must be tightly controlled

Policy gradient & beyond

Part 6

Black-box rewards: only a scalar per sample — REINFORCE to GRPO, then DiffusionNFT without policy gradient

REINFORCE (1/2): deriving the gradient

1 · GOAL: ∇Θ OF THE EXPECTED REWARD

$$J(\theta) = \mathbb{E}_{x\sim\pi_\theta(\cdot|c)}\big[R(x,c)\big] \;\approx\; \frac{1}{N}\sum_{i=1}^{N} R(x_i,c), \quad x_i \sim \pi_\theta(\cdot|c)$$

2 · PROBLEM: WE CAN'T BACKPROP

θ only shapes the distribution we sample from. Sampling x ~ π_θ has no gradient, and R may be a black box (OCR, detector, VLM)

3 · DERIVATION, STEP BY STEP

$$\nabla_\theta J(\theta) = \nabla_\theta \int \pi_\theta(x)\, R(x)\, dx$$

$$= \int \nabla_\theta \pi_\theta(x)\, R(x)\, dx$$

$$= \int \pi_\theta(x)\, \nabla_\theta \log \pi_\theta(x)\, R(x)\, dx$$

$$= \mathbb{E}_{x\sim\pi_\theta}\big[R(x)\, \nabla_\theta \log \pi_\theta(x)\big]$$

$$\approx \frac{1}{N}\sum_{i=1}^{N} R(x_i)\, \nabla_\theta \log \pi_\theta(x_i), \qquad x_i \sim \pi_\theta$$

① Write the expectation as an integral — θ sits only inside π_θ

② R does not depend on θ → move ∇ inside. But ∫∇π·R is not an expectation — nothing to sample from

③ **Log-derivative trick** — the π's cancel:

$$\nabla_\theta \log \pi_\theta = \frac{\nabla_\theta \pi_\theta}{\pi_\theta}$$

$$\Rightarrow \pi_\theta\, \nabla_\theta \log \pi_\theta = \cancel{\pi_\theta}\, \frac{\nabla_\theta \pi_\theta}{\cancel{\pi_\theta}} = \nabla_\theta \pi_\theta$$

④ π is back in front → an expectation over our own samples again

⑤ Monte-Carlo: sample N images and average. R is used only as a number

REINFORCE (2/2): what it does in practice

DLS

Sample $x_i \sim \pi_\theta$ → **Score $R(x_i)$** → **Loss $-R \cdot \log \pi_\theta$** → **Step θ, repeat**

IN CODE: WEIGHTED LOG-LIKELIHOOD

$$\mathcal{L}(\theta) = -\frac{1}{N}\sum_{i=1}^{N} R(x_i)\,\log \pi_\theta(x_i)$$

$R(x_i)$ is a constant (no gradient through R). $\nabla\mathcal{L}$ is exactly line ⑤ of the previous slide, with a minus sign

COMPARE WITH SFT

$$\mathcal{L}_{\mathrm{SFT}}(\theta) = -\frac{1}{N}\sum_{i=1}^{N} \log \pi_\theta(x_i), \quad x_i \sim \mathcal{D}$$

SFT = REINFORCE with $R \equiv 1$ on someone else's data.
REINFORCE: **own** samples, each weighted by its reward

Intuition

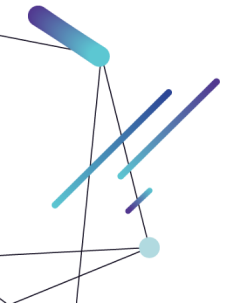
$R > 0 \rightarrow$ raise $\log \pi(x_i)$; $R < 0 \rightarrow$ lower it. Larger $|R| \rightarrow$ larger push. Good samples become more likely, bad ones less likely

The price: variance

If all rewards are positive (aesthetic score 5–7), every sample is pushed up; only noise separates good from bad. Fix: a baseline (next slide)

Catch for diffusion

$\log \pi_\theta(x_0|c)$ is intractable: x_0 is an integral over all denoising trajectories. RWR and DDPO (per-step MDP) work around it



Baselines for REINFORCE

DLS

WHAT: SUBTRACT B(C) → ADVANTAGE A

$$\nabla_\theta J = \mathbb{E}_{x\sim\pi_\theta}\Big[\underbrace{\big(R(x,c) - b(c)\big)}_{A(x,c)}\,\nabla_\theta \log \pi_\theta(x|c)\Big]$$

WHY IT STAYS UNBIASED

$$\mathbb{E}_{x\sim\pi_\theta}\big[\nabla_\theta \log \pi_\theta(x)\big] = \int \nabla_\theta \pi_\theta(x)\,dx = \nabla_\theta \int \pi_\theta(x)\,dx = \nabla_\theta 1 = 0$$

so E[b(c)·∇log π] = 0 for any b that does not depend on x

Why it cuts variance: an example

Three samples get R = 9.0, 9.5, 10.0.

Without b: all three are pushed *up* — the bad one only slightly less. The signal "which is better" is buried under the large common part.

With b = 9.5: A = −0.5, 0, +0.5 → the bad sample is pushed down, the good one up. Same expected gradient, far less noise.

Common choices of b(c)

- Running mean of rewards
- Per-prompt mean and std (DDPO)
- Mean of the G samples of the same prompt (GRPO)
- Learned value V(c) — a critic (PPO, actor–critic)

A good b ≈ E[R | c]: "how good is a typical sample for this prompt?"

Reward-weighted regression: the offline cousin of REINFORCE

DLS

SAME FORM: WEIGHT × ∇LOG Π — DIFFERENT SAMPLES AND WEIGHTS

$$\textbf{REINFORCE (online):}\quad \mathbb{E}_{x\sim\pi_\theta}\left[R(x,c)\,\boxed{\nabla_\theta \log \pi_\theta(x|c)}\right]$$

$$\textbf{RWR (offline):}\quad \mathbb{E}_{x\sim\mathcal{D}}\left[e^{R(x,c)/\beta}\,\boxed{\nabla_\theta \log \pi_\theta(x|c)}\right]$$

FOR DIFFUSION: LOG Π REPLACED BY THE WEIGHTED DENOISING LOSS (ELBO)

$$\mathcal{L}_{\mathrm{RWR}}(\theta) = \mathbb{E}_{c,\, x_0\sim p_{\mathrm{old}}}\left[\, w(x_0,c)\,\boxed{\mathbb{E}_{t,\epsilon}\left\|\epsilon-\epsilon_\theta(x_t,t,c)\right\|^2}\,\right], \qquad w=\exp(r/\beta)$$

Red boxes play the same role: −MSE ≈ log π (ELBO), so ∇ of the MSE term ≈ −∇log π

REINFORCE is online: fresh samples from the current π_θ after every update

Weights exp(R/β) ≥ 0: the optimum is π* ∝ π_old · exp(R/β) — weighted SFT

RWR is offline (as posed for T2I): generate once, label, then do weighted fine-tuning on that fixed set

Weakness: can't push bad samples down; DDPO beats it as a baseline

Peters & Schaal, ICML 2007 · Lee et al. — arXiv:2302.12192 · Black et al. (DDPO, RWR baseline) — arXiv:2305.13301

PPO, step 1: importance sampling

Problem: REINFORCE is on-policy. After one gradient step π_θ has changed, so the samples are stale and must be thrown away. For diffusion that is expensive: every image costs 20–50 network calls.

IMPORTANCE-SAMPLING IDENTITY

$$\mathbb{E}_{x\sim\pi_\theta}\big[f(x)\big] = \mathbb{E}_{x\sim\pi_{\text{old}}}\left[\frac{\pi_\theta(x)}{\pi_{\text{old}}(x)}\, f(x)\right]$$

Idea

Sample once from π_old, then correct for the fact that we now train π_θ: re-weight each sample by how much more (or less) likely π_θ makes it.

SURROGATE OBJECTIVE — ITS GRADIENT AT Θ = Θ_OLD IS EXACTLY THE POLICY GRADIENT

$$\mathcal{L}^{\text{IS}}(\theta) = \mathbb{E}_{x\sim\pi_{\text{old}}}\big[\rho_\theta(x)\, A(x)\big], \qquad \rho_\theta(x) = \frac{\pi_\theta(x|c)}{\pi_{\text{old}}(x|c)}, \qquad \nabla_\theta \mathcal{L}^{\text{IS}}\Big|_{\theta=\theta_{\text{old}}} = \nabla_\theta J$$

Gain

One batch of rollouts → several gradient steps (epochs)

Danger

If π_θ drifts far from π_old, ρ explodes or vanishes → huge variance, the surrogate stops being accurate

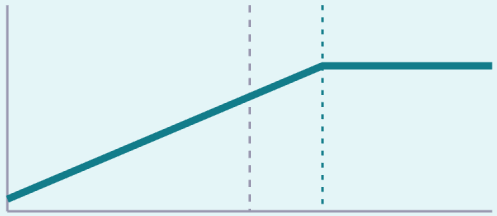
PPO, step 2: clip the ratio

DLS

PPO CLIPPED OBJECTIVE (TYPICAL E = 0.1–0.2)

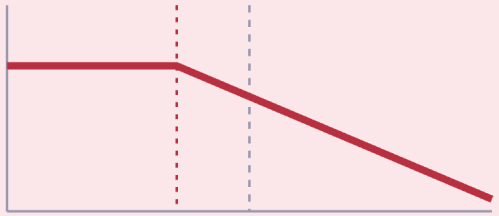
$$\mathcal{J}^{\mathrm{PPO}}(\theta) = \mathbb{E}_{x\sim\pi_{\mathrm{old}}}\Big[\min\big(\rho\,A,\ \mathrm{clip}(\rho, 1-\varepsilon, 1+\varepsilon)\,A\big)\Big], \qquad \rho = \frac{\pi_\theta(x|c)}{\pi_{\mathrm{old}}(x|c)}$$

Good sample, A > 0



We want ρ ↑ (make it more likely). The objective grows only until ρ = 1 + ε, then is flat → no gradient to push further.

Bad sample, A < 0



We want ρ ↓ (make it less likely). The objective stops rewarding the push once ρ = 1 − ε.

Why min?

Pessimistic choice: clipping removes only the incentive for big improving moves, never hides a step that makes things worse.

In practice

Several epochs per batch; A from a critic (PPO) or the group (GRPO); often + KL.

x-axis: ratio ρ (dashed grey: ρ = 1; coloured dashed: 1 ± ε) · y-axis: objective · Schulman et al., PPO — arXiv:1707.06347

Denoising as a Markov decision process

REINFORCE OVER THE DENOISING TRAJECTORY

$$J(\theta) = \mathbb{E}_{c,\, x_{T:0}\sim p_\theta}\big[r(x_0, c)\big], \qquad x_T \sim \mathcal{N}(0, I), \qquad x_{t-1} \sim p_\theta(\cdot \mid x_t, c)$$

$$\log p_\theta(x_{T:0} \mid c) = \underbrace{\log p(x_T)}_{\text{no } \theta} + \sum_{t=1}^{T} \log p_\theta(x_{t-1} \mid x_t, c) \quad \longrightarrow \quad \nabla_\theta J = \mathbb{E}\Big[r(x_0, c) \sum_{t=1}^{T} \nabla_\theta \log p_\theta(x_{t-1} \mid x_t, c)\Big]$$

ONE STOCHASTIC SAMPLER STEP — GENERIC A_t, Σ_t

$$p_\theta(x_{t-1} \mid x_t, c) = \mathcal{N}\big(\mu_\theta,\, g_t^2 I\big), \quad \mu_\theta = \alpha_{t-1}\hat{x}_0 + \sqrt{\sigma_{t-1}^2 - g_t^2}\,\hat{\varepsilon}, \quad \hat{x}_0 = \frac{x_t - \sigma_t \hat{\varepsilon}}{\alpha_t}, \quad \hat{\varepsilon} = \varepsilon_\theta(x_t, t, c)$$

$$\log p_\theta(x_{t-1}|x_t, c) = -\frac{\big\|x_{t-1} - \mu_\theta(x_t, t, c)\big\|^2}{2g_t^2} + \text{const}$$

MDP (DDPO): state (c, t, x_t), action x_{t-1}, policy $p_\theta(x_{t-1} \mid x_t, c)$

Sparse reward: only x_0 is scored → credit assignment across T steps

Needs stochasticity: deterministic ODE / DDIM has $g_t = 0$ → no density

DDPO: REINFORCE and PPO on the denoising MDP

DLS

DDPO-SF = REINFORCE, SUMMED OVER DENOISING STEPS

$$\nabla_\theta J(\theta) = \mathbb{E}_{c,\, x_{0:T}\sim p_\theta}\left[r(x_0, c) \sum_{t=1}^{T} \nabla_\theta \log p_\theta(x_{t-1}|x_t, c)\right]$$

DDPO-IS = PPO, WITH A PER-STEP LIKELIHOOD RATIO

$$\mathcal{J}^{\text{clip}} = \mathbb{E}\Big[\sum_t \min\big(\rho_t A,\ \text{clip}(\rho_t, 1-\varepsilon, 1+\varepsilon)\, A\big)\Big], \qquad \rho_t = \frac{p_\theta(x_{t-1}|x_t, c)}{p_{\theta_{\text{old}}}(x_{t-1}|x_t, c)}$$

Black-box reward

Only needs $r(x_0, c)$ as a number — VLM judges, OCR, detectors all work

Advantage

Per-prompt reward normalization as a baseline reduces variance

Costs

Stochastic sampler, high variance, many rollouts; DPOK adds KL

DPO recap (LLMs)

1 · Optimum of KL-RL
$p^* \propto p_{ref} \cdot \exp(r/\beta)$

→

2 · Invert for reward
$r = \beta \log(p^*/p_{ref}) + \beta \log Z$

→

3 · Bradley–Terry
$Z(c)$ cancels in $r(x^w) - r(x^l)$

→

4 · Parametrize p^*
logistic loss on pairs

IMPLICIT REWARD

$$r(x, c) = \beta \log \frac{p^*(x|c)}{p_{\text{ref}}(x|c)} + \beta \log Z(c)$$

DPO LOSS

$$\mathcal{L}_{\text{DPO}} = -\,\mathbb{E}\, \log \sigma\left(\beta \log \frac{\pi_\theta(y^w|x)}{\pi_{\text{ref}}(y^w|x)} - \beta \log \frac{\pi_\theta(y^l|x)}{\pi_{\text{ref}}(y^l|x)}\right)$$

INTUITION

The model is its own reward model: $\hat{r}(x) = \beta \log \pi_\theta(x)/\pi_{ref}(x)$ — "how much more likely do I make x than the reference?"

The loss is logistic regression on pairs: rank the winner above the loser → raise the winner's likelihood relative to ref, lower the loser's

No sampling, no reward model: the gradient is largest on pairs the model still ranks wrong; β sets how far from ref we may go

Problem for diffusion: $p(x_0|c)$ is intractable — the log-ratios cannot be evaluated directly.

Diffusion-DPO: from images to trajectories

DLS

STEP 1 — WORK WITH THE WHOLE CHAIN

$$\log \frac{p_\theta(x_0|c)}{p_{\text{ref}}(x_0|c)} \longrightarrow \mathbb{E}_{x_{1:T}}\left[\sum_{t=1}^{T} \log \frac{p_\theta(x_{t-1}|x_t,c)}{p_{\text{ref}}(x_{t-1}|x_t,c)}\right]$$

2 · Jensen bound

Move the expectation over trajectories outside the log σ → an upper bound we minimize with one sampled t

3 · Forward instead of reverse

Replace the model's reverse chain by the forward noising process q: just add noise to x^w and x^l

4 · Gaussian KL = MSE

Per-step KL between Gaussians reduces to a squared denoising error $\|\varepsilon - \hat{\varepsilon}\|^2$, weighted by $\omega(\lambda_t)$

Net effect: a DPO loss that only needs noised versions of the two images and one denoiser call each for θ and the reference.

Diffusion-DPO: the final loss

DLS

DIFFUSION-DPO

$$\mathcal{L}_{\text{Diff-DPO}}(\theta) = -\mathbb{E}\log\sigma\Big(-\beta T\omega(\lambda_t)\Big[\|\epsilon^w - \epsilon_\theta(x_t^w, t)\|^2 - \|\epsilon^w - \epsilon_{\text{ref}}(x_t^w, t)\|^2 - \big(\|\epsilon^l - \epsilon_\theta(x_t^l, t)\|^2 - \|\epsilon^l - \epsilon_{\text{ref}}(x_t^l, t)\|^2\big)\Big]\Big)$$

Training step

1. Sample pair (c, x^w, x^l), timestep t, noises ε^w, ε^l
2. Noise both images with the forward process
3. 4 denoiser calls: θ and ref on winner and loser
4. Logistic loss on the difference of error differences

Facts & caveats

Data: Pick-a-Pic v2, ~851k preference pairs

Model: SDXL-1.0; preferred over base SDXL by human raters

Offline: pairs from older models → distribution shift

Large β: ≈ 2500–5000, because the loss is a per-pixel MSE

GRPO

GROUP-RELATIVE ADVANTAGE

$$A_i = \frac{r_i - \text{mean}\left(\{r_j\}_{j=1}^{G}\right)}{\text{std}\left(\{r_j\}_{j=1}^{G}\right)}$$

INTUITION

For each prompt, generate G images and let them **compete with each other**: better than the group average → push up, worse → push down. The siblings are the baseline, so no value network is needed. Dividing by the group std makes the step the same size for easy and hard prompts, whatever the reward scale. PPO clipping + KL keep each step small.

One prompt → G = 7 samples → rewards → advantages (illustrative)


+1.0


+1.2


−0.3


−0.9


+0.8


−0.3


−1.5

GRPO OBJECTIVE

$$\mathcal{J}_{\text{GRPO}}(\theta) = \mathbb{E}\Big[\frac{1}{G}\sum_{i=1}^{G}\frac{1}{T}\sum_{t}\min\big(\rho_t^i A_i,\ \text{clip}(\rho_t^i, 1-\varepsilon, 1+\varepsilon)A_i\big) - \beta\,\text{KL}\big(\pi_\theta \,\|\, \pi_{\text{ref}}\big)\Big]$$

Flow-GRPO: turning the ODE into an SDE

DLS

MARGINAL-PRESERVING SDE (SAME P_t AS THE ODE)

$$dx_t = \left[v_\theta(x_t, t) - \frac{g_t^2}{2}\nabla_x \log p_t(x_t)\right]dt + g_t\, d\bar{w}_t, \qquad \nabla_x \log p_t(x) = \frac{\dot{\alpha}_t\, x - \alpha_t\, v_\theta(x,t)}{\sigma_t\,(\alpha_t\dot{\sigma}_t - \dot{\alpha}_t\sigma_t)}$$

POLICY = EULER–MARUYAMA STEP

$$\pi_\theta(x_{t-\Delta t} \mid x_t, c) = \mathcal{N}\big(\mu_\theta(x_t, t, c),\ g_t^2\,\Delta t\ I\big)$$

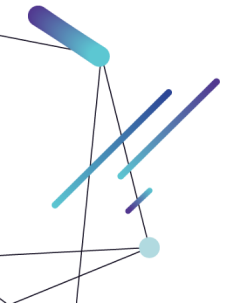
CLOSED-FORM PER-STEP KL

$$\mathrm{KL}\big(\pi_\theta \| \pi_{\mathrm{ref}}\big) = \frac{\|\mu_\theta - \mu_{\mathrm{ref}}\|^2}{2\,g_t^2\,\Delta t} \ \propto\ \big\|v_\theta - v_{\mathrm{ref}}\big\|^2$$

Exploration: the noise scale g_t (a hyper-parameter) injects stochasticity; the score comes from v_θ via α_t, σ_t

Same marginals: SDE and ODE sample the same distribution — inference can still use the ODE

Exact likelihood ratios: ρ_t is a ratio of two Gaussians with the same variance



Flow-GRPO: training tricks and results

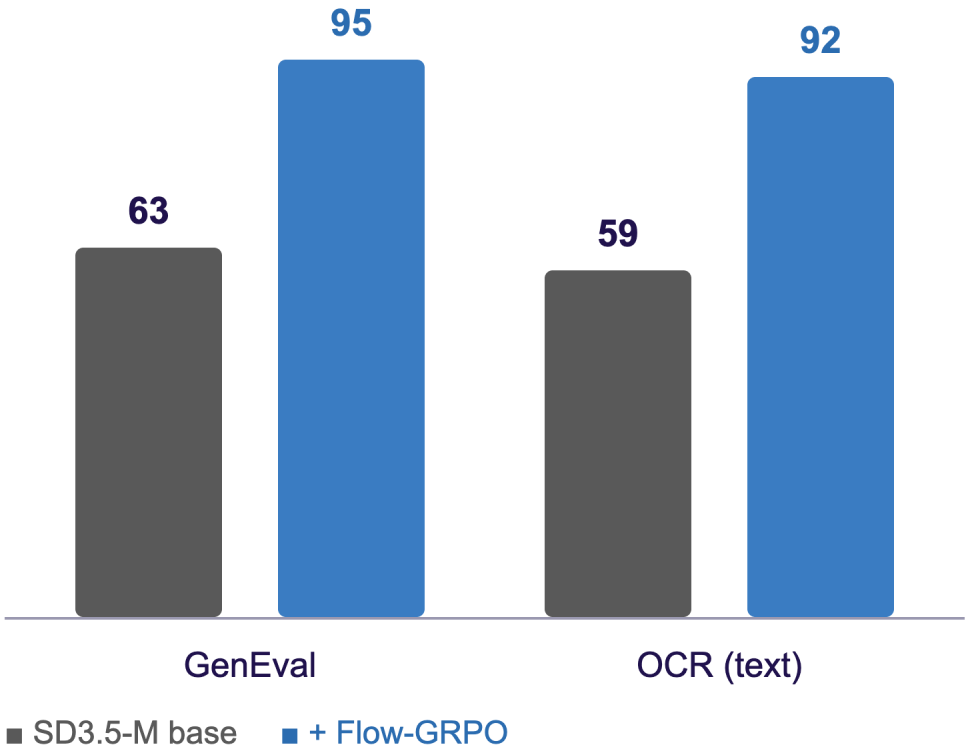
Denoising reduction: train with ~10 sampling steps, infer with ~40 → much cheaper rollouts, little loss in final quality

KL to reference: keeps image quality and diversity; without it rewards are hacked

Group size: many images per prompt for reliable advantages

Rewards: GenEval (rule-based detector), OCR, PickScore

Accuracy on SD3.5-Medium (%)



Limits of SDE-based policy gradient

DLS

Solver lock-in

The policy is one specific SDE discretization; training and sampler are coupled

SDE samples are worse

Injected noise lowers sample quality vs. the ODE the model ships with

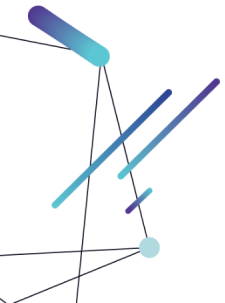
Long credit assignment

One scalar reward spread over all T steps; per-step ratios add variance

CFG in the loop

Guided rollouts double the cost and complicate the likelihood

Question: can we do online RL with only the standard flow-matching loss on the forward process — no likelihoods, any sampler?



DiffusionNFT: online RL on the forward process

DLS

IMPLICIT POSITIVE / NEGATIVE POLICIES

$$v_\theta^+ = (1-\beta)\, v^{\text{old}} + \beta\, v_\theta, \qquad v_\theta^- = (1+\beta)\, v^{\text{old}} - \beta\, v_\theta$$

REWARD → OPTIMALITY PROBABILITY

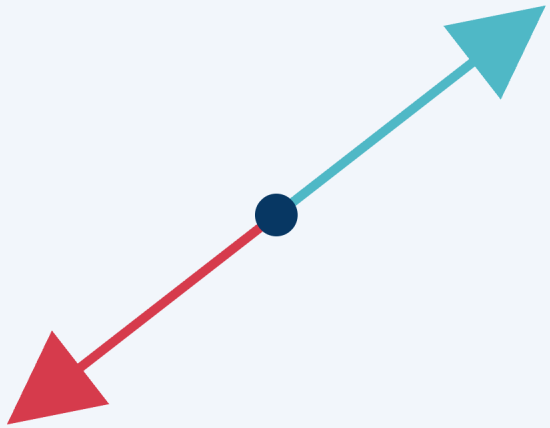
$$r = \frac{1}{2} + \frac{1}{2}\,\text{clip}\left(\frac{r^{\text{raw}} - \mathbb{E}_{\pi^{\text{old}}}[r^{\text{raw}}|c]}{Z_c},\, -1,\, 1\right) \in [0,1]$$

Forward process: loss on noised clean images, like pretraining — no trajectories stored

Positive & negative: bad samples are used too, unlike SFT/RWR

Off-policy-friendly: the old velocity is a slowly updated copy of the model

Geometry of the update



Teal: v^+ moves toward positives

Red: v^- mirrors it; fitting negatives with v^- pushes the model away from them

DiffusionNFT: objective and results

DIFFUSIONNFT LOSS

$$\mathcal{L}_{\mathrm{NFT}}(\theta) = \mathbb{E}_{c,\, x_0 \sim \pi^{\mathrm{old}},\, t,\, \epsilon}\left[r \left\| v_\theta^+(x_t, c, t) - v \right\|^2 + (1 - r) \left\| v_\theta^-(x_t, c, t) - v \right\|^2 \right]$$

OPTIMUM = RL AS LEARNED GUIDANCE

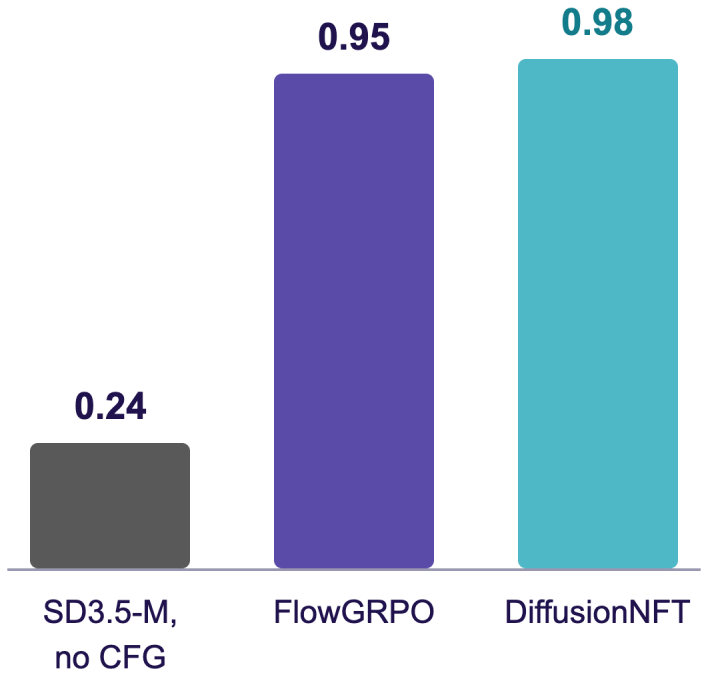
$$v_\theta^* = v^{\mathrm{old}} + \frac{2}{\beta} \Delta \qquad (\Delta = \text{reinforcement guidance direction})$$

Solver-agnostic: collect data with any black-box sampler (ODE worked best)

CFG-free: starts unguided and ends above the CFG baseline

Efficiency: 3×–25× faster than FlowGRPO in wall-clock

GenEval score



FlowGRPO: >5k iterations, with CFG.
DiffusionNFT: ~1k iterations, CFG-free.

Method landscape

Method	Online	Differentiable r	Needs likelihood	Sampler in training	Signal
SFT / RWR	No / opt.	No	No	— / any	Curated or reweighted data
Diffusion-DPO	No	No	ELBO proxy	—	Preference pairs
ReFL	Yes	Yes	No	Any (1 step w/ grad)	∇r at $\hat{x}_0$
DRaFT / -K / -LV	Yes	Yes	No	Differentiable (DDIM)	∇r through K steps
DDPO / DPOK	Yes	No	Per-step Gaussian	Stochastic ($\eta>0$)	Scalar r + PPO
Flow-GRPO	Yes	No	Per-step Gaussian	SDE	Group advantage + KL
DiffusionNFT	Yes	No	No	Any (ODE best)	Pos./neg. FM loss
OPD	Yes	No (teachers)	No	ODE (student)	Per-step velocity regression to teacher

Merging several rewards

Part 7

On-policy distillation (OPD) of reward-specific experts into one model

Multi-reward alignment: the seesaw problem

DLS

NAIVE: WEIGHTED SUM

$$r_{\text{mix}}(x_0, c) = \sum_{k=1}^{K} w_k \, r_k(x_0, c)$$

Seesaw example: single-reward GRPO on PickScore lowered GenEval to 0.51 (Flow-OPD)

Goal: one model ≈ the best expert on every reward

Critic

Value

1 · Weighted sum in one RL run

Gradient interference and scale imbalance → seesaw; weights hard to tune

2 · Cascade RL

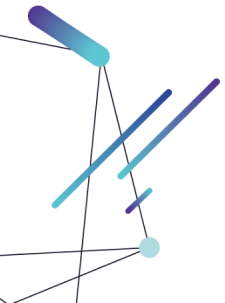
One reward after another → catastrophic forgetting; long pipelines

3 · Weight merging / soups

Cheap, but conflicting task vectors interfere

4 · On-policy distillation (OPD)

Train experts separately, distill into one student with dense per-step supervision



On-Policy Distillation (OPD) for diffusion

GenEval expert

OCR expert

PickScore expert

single-reward Flow-GRPO teachers

→ **Cold start**
SFT on teacher samples or merged weights

→ **Student rollout**
own deterministic ODE trajectory, x_T → x_0

→ **Route + score**
teacher T(c) evaluates student's states

→ **Dense regression**
match teacher velocity at every step

OPD OBJECTIVE — VELOCITY REGRESSION ON STUDENT-VISITED STATES

1 · Student rollout: deterministic ODE (Euler), same sampler as at inference, no gradient

$$x_{t_{j-1}} = \mathrm{sg}\big[\, x_{t_j} + (t_{j-1} - t_j)\, v_\theta(x_{t_j}, t_j, c)\,\big], \qquad x_{t_N} = x_T \sim \mathcal{N}(0, I)$$

2 · On every visited state, regress the student's velocity onto the routed teacher's

$$\mathcal{L}_{\mathrm{OPD}}(\theta) = \mathbb{E}_{c,\, x_T}\Big[\sum_{j=1}^{N} \tfrac{1}{2}\,\Delta t_j^2\, \big\| v_\theta(x_{t_j}, t_j, c) - \mathrm{sg}\big(v_{T(c)}(x_{t_j}, t_j, c)\big)\big\|^2\Big] = \mathbb{E}\Big[\sum_{j} \tfrac{1}{2}\|\mu_\theta - \mu_{T(c)}\|^2\Big]$$

Why on-policy distillation works

Dense supervision

A target velocity at every step, not one scalar at the end

Low variance

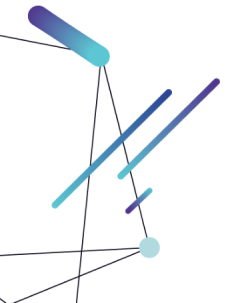
Closed-form regression — no score-function / likelihood-ratio term

On-policy

The teacher corrects the student where it actually goes → no exposure bias

No interference

Each prompt gets one teacher; experts never fight inside one objective



Flow-OPD results on SD3.5-Medium

Model	GenEval ↑	OCR ↑	DeQA ↑	PickScore ↑	Avg ↑
SD3.5-M (base)	0.63	0.59	4.07	21.64	0.717
GRPO-Mix	0.73	0.83	4.33	21.84	0.817
SFT + GRPO-Mix	0.85	0.86	4.29	21.79	0.852
Flow-OPD (SFT cold start)	0.91	0.92	4.29	21.83	0.882
Flow-OPD (merge cold start)	0.93	0.93	4.31	23.05	0.902

Merge + OPD beats SFT + OPD

Weight merging is a good cold start — soups and OPD compose

No seesaw

GRPO on PickScore alone drops GenEval to 0.51; OPD keeps all metrics high

A practical alignment recipe

DLS

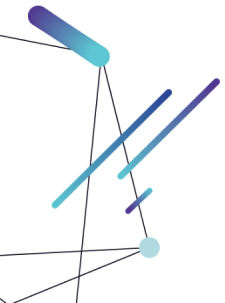
1	2	3	4	5	6
Pretrain	**SFT**	**Soup**	**Online RL**	**Distill**	**Evaluate**
web-scale flow or diffusion model	quality tuning, recaptioning	merge SFT variants (optional)	Flow-GRPO / NFT, one expert per reward, KL to ref	OPD of experts into one student	held-out RMs, GenEval, OCR, human eval

Differentiable reward & need speed: DRaFT-LV / ReFL with LoRA and early stopping

Only preference pairs: Diffusion-DPO (then iterate online if possible)

Black-box reward (OCR, detector, VLM): Flow-GRPO, or DiffusionNFT for efficiency

Many rewards: separate experts + OPD, merged-weights cold start



References

- Dai et al. Emu: Enhancing Image Generation Models Using Photogenic Needles in a Haystack, 2023
- Wortsman et al. Model soups, ICML 2022 · Ramé et al. Rewarded soups, NeurIPS 2023
- Black et al. Training Diffusion Models with RL (DDPO), ICLR 2024 · Fan et al. DPOK, NeurIPS 2023
- Xu et al. ImageReward / ReFL, NeurIPS 2023 · Kirstain et al. Pick-a-Pic / PickScore, NeurIPS 2023
- Wu et al. Human Preference Score v2, 2023 · Ma et al. HPSv3, ICCV 2025
- Clark et al. Directly Fine-Tuning Diffusion Models on Differentiable Rewards (DRaFT), ICLR 2024
- Wallace et al. Diffusion Model Alignment Using Direct Preference Optimization, CVPR 2024
- Shao et al. DeepSeekMath (GRPO), 2024 · Liu et al. Flow-GRPO, NeurIPS 2025
- Zheng et al. DiffusionNFT: Online Diffusion Reinforcement with Forward Process, ICLR 2026
- Fang et al. Flow-OPD: On-Policy Distillation for Flow Matching Models, arXiv 2605.08063
- Li et al. DiffusionOPD: A Unified Perspective of On-Policy Distillation in Diffusion Models, arXiv 2605.15055